

Topic: Live API Testing with [TheCatAPI.com](https://api.thecatapi.com)

Tools: Postman

Instructions:

- Use the base URL: <https://api.thecatapi.com/v1>

Q1. What are the key components of a REST API response you should validate during manual testing?

Answer:

Key Components of a REST API Response

- **HTTP Status Codes:** The tester validates the status code to confirm the request's outcome, such as "200 OK" for success or "404 Not Found" for missing resources.
- **Response Body:** The data within the payload is verified to ensure it is accurate, complete, and adheres to the expected format (e.g. JSON).
- **Response Headers:** Metadata such as "Content-Type" and authorization details are inspected to confirm the server is communicating according to the defined protocol.

Q2. List four types of API test cases you should write when testing a public API like TheCatAPI.

Answer:

Types of API Test Cases

- **Functional Test Cases:** These are utilized to verify that the API returns the correct and expected responses when provided with valid input data.
- **Negative Test Cases:** These evaluate how the system handles invalid, malformed, or missing inputs.
- **Performance Test Cases:** The tester measures response times and system behavior under various load conditions to ensure scalability and speed.
- **Security Test Cases:** These focus on ensuring proper authentication and authorization mechanisms to protect sensitive data.

Q3. Why is it important to test API behavior when required parameters are missing or invalid?

Answer:

Testing missing or invalid parameters ensures the API can handle errors properly without crashing. It helps verify that correct error messages and status codes (like 400 Bad Request) are returned. This also improves security by preventing misuse of the API. Overall, it makes the system more reliable and user-friendly.

Q4.

Design a test case for the following API:

GET /images/search?limit=2&mime_types=jpg

Answer:

Test Case ID: TC_API_01

Summary: Verify that the API returns 2 JPG images

Pre-condition:

- API is accessible
- Internet connection available

Steps:

1. Send GET request:
https://api.thecatapi.com/v1/images/search?limit=2&mime_types=jpg
2. Observe the response

Expected Result:

- API returns exactly 2 images
- All images are in JPG format

Status Code:

- 200 OK

Validation Logic:

- Response is JSON array
- Array length = 2
- Each object contains fields like **id**, **url**, **height**, **width**
- Image URLs end with **.jpg** or MIME type = jpg

Q5.

Design a **negative test case** for this API:

GET /breeds/search?q=abcxyz

Answer:

Test Case ID: TC_API_02

Summary: Verify that the API handles invalid search query

Pre-condition:

- API is accessible
- Internet connection available

Steps:

1. Send GET request: <https://api.thecatapi.com/v1/breeds/search?q=abcxyz>
2. Check response

Expected Result:

- API returns empty array `[]`
- No error or crash

Status Code:

- 200 OK

Validation Logic:

- Response is valid JSON
- Array is empty
- No unexpected fields or errors

Q6.

Write one test case for adding a favorite cat image using this API:

POST /favourites

Use your own API key and include a valid image_id from /images/search.

Answer:

Test Case ID: TC_API_03

Summary: Verify user can add a favorite image

Pre-condition:

- Valid API key available
- Valid **image_id** from <https://api.thecatapi.com/v1/images/search>

Steps:

1. Send POST request <https://api.thecatapi.com/v1/favourites>
2. Add headers: **x-api-key**
3. Body:

```
{  
  "image_id": "vYfvpfuU7"  
}
```

Expected Result:

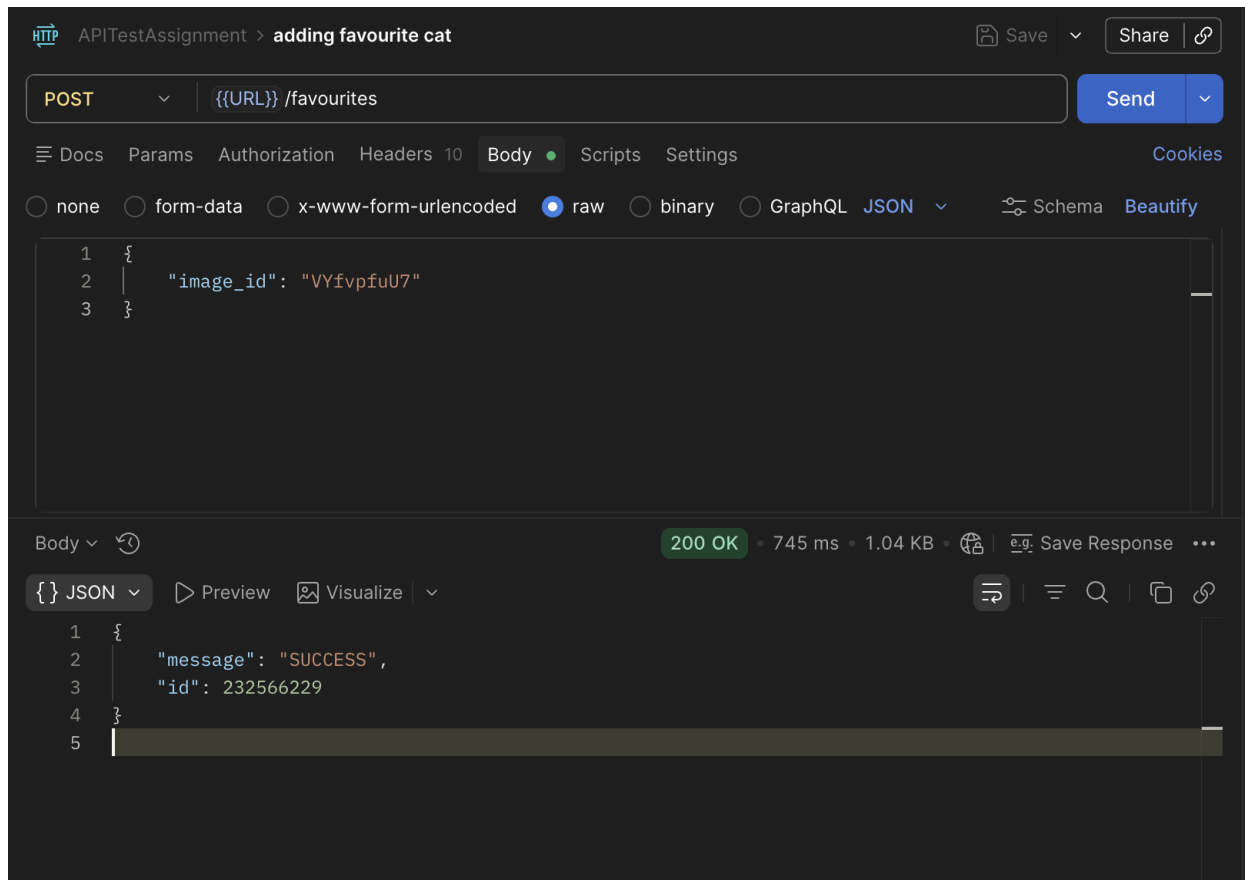
- Favorite is successfully added
- Response contains **message, id**

Status Code:

- 200 OK

Validation Logic:

- Response contains correct **id**
- Favourite ID is generated
- Data is stored successfully



Q7.

Write a test case for deleting a favorite using this API:

DELETE /favourites/{favourite_id}

Describe how you would validate the test even if the image was not in the favorite list before.

Answer:

Test Case ID: TC_API_04

Summary: Verify that a favorite image can be deleted successfully

Pre-condition:

- Valid API key is available
- A valid or invalid **favourite_id** is known

Steps:

1. Send DELETE request to
https://api.thecatapi.com/v1/favourites/{favourite_id}
2. Include header: **x-api-key**
3. Observe the response

Expected Result:

- If the favourite exists → it is deleted successfully
- If the favourite does not exist → API handles it gracefully without crashing

Status Code:

- 200 OK (for successful deletion)
- 404 Not Found or 200 with message (if not existing, depending on API design)

Validation Logic:

- **For existing favourite:**
 - Verify the response indicates success
 - Call GET <https://api.thecatapi.com/v1/favourites> to confirm the item is no longer present
- **For non-existing favourite:**
 - Ensure API does not return server error (no 500)
 - Response should be safe (e.g., 404 or empty success response)
 - System remains stable and consistent

Q8.

Execute the following request and paste the response:

GET /breeds/search?q=sib

Then write what validations you would perform on the response.

Answer:

- Request: GET {{URL}}/breeds/search?q=sib
- Response: [

```
[
  {
    "weight": {
```

```
•         "imperial": "8 - 16",
•         "metric": "4 - 7"
•     },
•     "id": "sibe",
•     "name": "Siberian",
•     "breed_group": null,
•     "cfa_url": "http://cfa.org/Breeds/BreedsSthruT/Siberian.aspx",
•     "vetstreet_url": "http://www.vetstreet.com/cats/siberian",
•     "vcahospitals_url":
"https://vcahospitals.com/know-your-pet/cat-breeds/siberian",
•     "temperament": "Curious, Intelligent, Loyal, Sweet, Agile, Playful,
Affectionate",
•     "origin": "Russia",
•     "country_codes": "RU",
•     "country_code": "RU",
•     "description": "The Siberians dog like temperament and affection makes
the ideal lap cat and will live quite happily indoors. Very agile and powerful,
the Siberian cat can easily leap and reach high places, including the tops of
refrigerators and even doors. ",
•     "life_span": "12 - 15",
•     "indoor": 0,
•     "lap": 1,
•     "alt_names": "Moscow Semi-longhair, HairSiberian Forest Cat",
•     "adaptability": 5,
•     "affection_level": 5,
•     "child_friendly": 4,
•     "dog_friendly": 5,
•     "energy_level": 5,
•     "grooming": 2,
•     "health_issues": 2,
•     "intelligence": 5,
•     "shedding_level": 3,
```

```

•      "social_needs": 4,
•      "stranger_friendly": 3,
•      "vocalisation": 1,
•      "experimental": 0,
•      "hairless": 0,
•      "natural": 1,
•      "rare": 0,
•      "rex": 0,
•      "suppressed_tail": 0,
•      "short_legs": 0,
•      "wikipedia_url": "https://en.wikipedia.org/wiki/Siberian_(cat)",
•      "hypoallergenic": 1,
•      "reference_image_id": "3bkZAJRh1"
•    }
•  ]
•

```

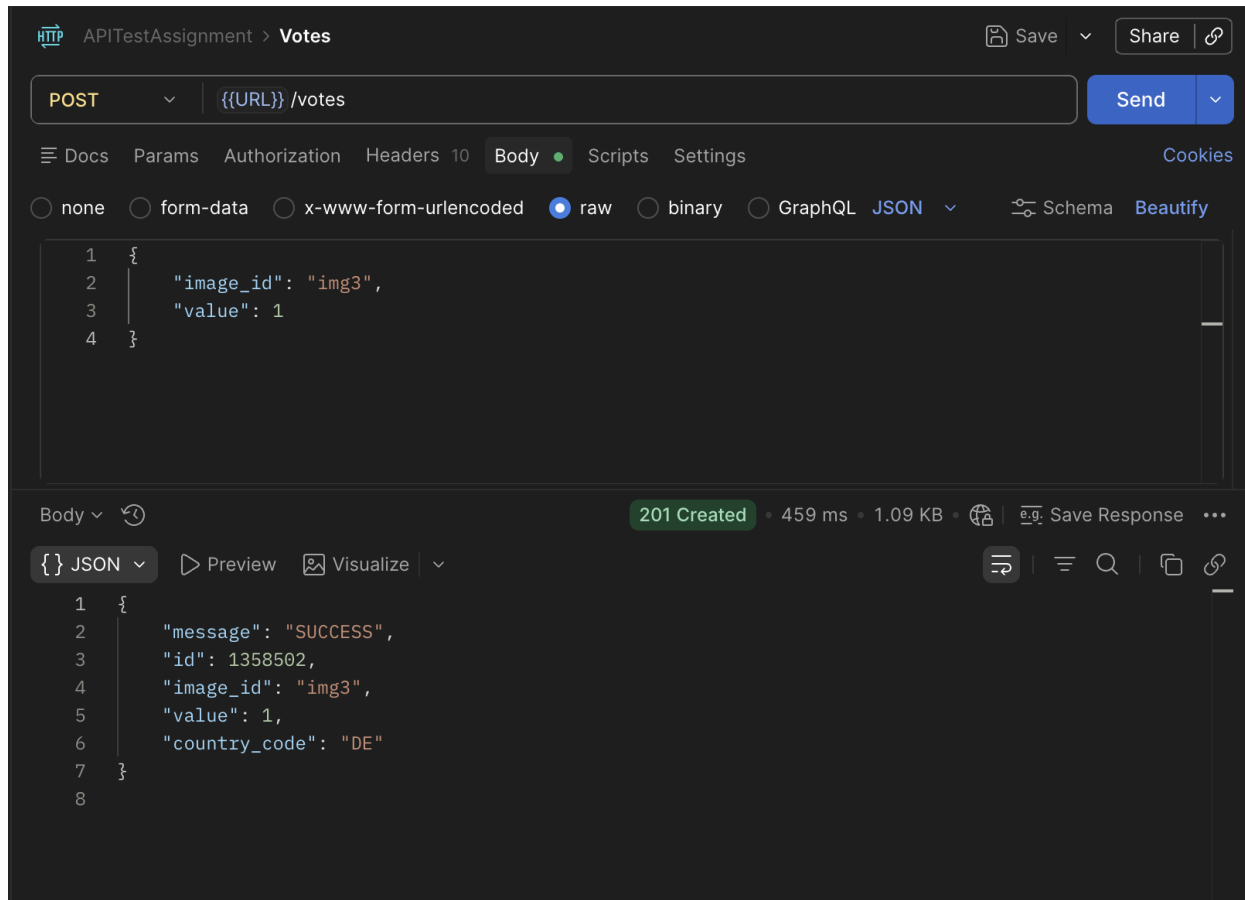
- After executing the request, I would validate the status code, response format, and correctness of returned data. I would ensure the response contains relevant results matching the search query and verify required fields are present with correct data types. I would also check how the API behaves in edge cases such as no results.

Q9.

Vote for a cat image (value = 1) using your API key.

Show the exact request sent and the full JSON response received.

Answer:



Q10.

Search for a cat image with this request:
GET /images/search?mime_types=gif&limit=1
Write a Postman test script that validates:

- It returns 1 result
- The image URL ends with .gif

Answer:

- Script:

```
pm.test("Response returns exactly 1 result", function () {  
  
    pm.expect(pm.response.json()).to.be.an('array');
```

```

    pm.expect(pm.response.json().length).to.eql(1);

});

pm.test("The image URL has a .gif extension", function () {

    pm.expect(pm.response.json()[0].url).to.match(/\.gif$/);

});

```

- **Test Results:**

The screenshot shows the Postman interface for a REST client request. The request is a GET method to the URL `{{URL}}/images/search?mime_types=gif&limit=1`. The 'Scripts' tab is active, displaying the following JavaScript code:

```

1 pm.test("Response returns exactly 1 result", function () {
2   pm.expect(pm.response.json()).to.be.an('array');
3   pm.expect(pm.response.json().length).to.eql(1);
4 });
5
6 pm.test("The image URL has a .gif extension", function () {
7   pm.expect(pm.response.json()[0].url).to.match(/\.gif$/);
8 });

```

Below the script editor, the 'Test Results' section shows two tests that have passed:

- PASSED** Response returns exactly 1 result
- PASSED** The image URL has a .gif extension

The status bar at the top indicates a 200 OK response with a response time of 2.65 s and a size of 1.17 KB.

Q11.

Check how the API responds to an invalid limit value.

Send:

GET /images/search?limit=1000

Share:

- Status code
- Length of the response
- Your analysis: Is the API properly validating the limit?

Answer:

Test Results

- Status Code: 200
- Length of the Response: 10

Is the API properly validating the limit?

Yes, the API is validating the limit, but it does so by adjusting the value instead of rejecting it. When a user requests a very high limit (like 1000), the API automatically reduces it to a safe maximum (e.g., 10) by following sanitization approach and helps keep the system stable and fast. Instead of showing an error, it gives a valid response, which improves user experience. It also protects the database from heavy load and performance issues.



APITestAssignment > search_images



Save



Share



GET



{{URL}} /images/search?limit=1000

Send



Docs

Params

Authorization

Headers 7

Body

Scripts

Settings

Cookies

Query Params

Key	Value	Description	Bulk Edit	...
☒ limit	1000			

Body



200 OK

•

263 ms

•

2 KB



Save Response

...

{ } JSON

Preview



Visualize



	id	url	width	height
0	442	https://cdn2.thecatapi.com/images/442.gif	498	258
1	abq	https://cdn2.thecatapi.com/images/abq.gif	500	375
2	b1i	https://cdn2.thecatapi.com/images/b1i.jpg	526	620
3	bmu	https://cdn2.thecatapi.com/images/bmu.jpg	1000	695
4	dpq	https://cdn2.thecatapi.com/images/dpq.jpg	600	804
5	eg1	https://cdn2.thecatapi.com/images/eg1.jpg	1600	1200
6	MTY4OTA3OQ	https://cdn2.thecatapi.com/images/MTY4OTA3OQ.jpg	900	593
7	MTgzMzl1Mw	https://cdn2.thecatapi.com/images/MTgzMzl1Mw.jpg	500	424
8	--ovPy5Lb	https://cdn2.thecatapi.com/images/--ovPy5Lb.jpg	680	1024
9	clFwPWY84	https://cdn2.thecatapi.com/images/clFwPWY84.jpg	1705	1280